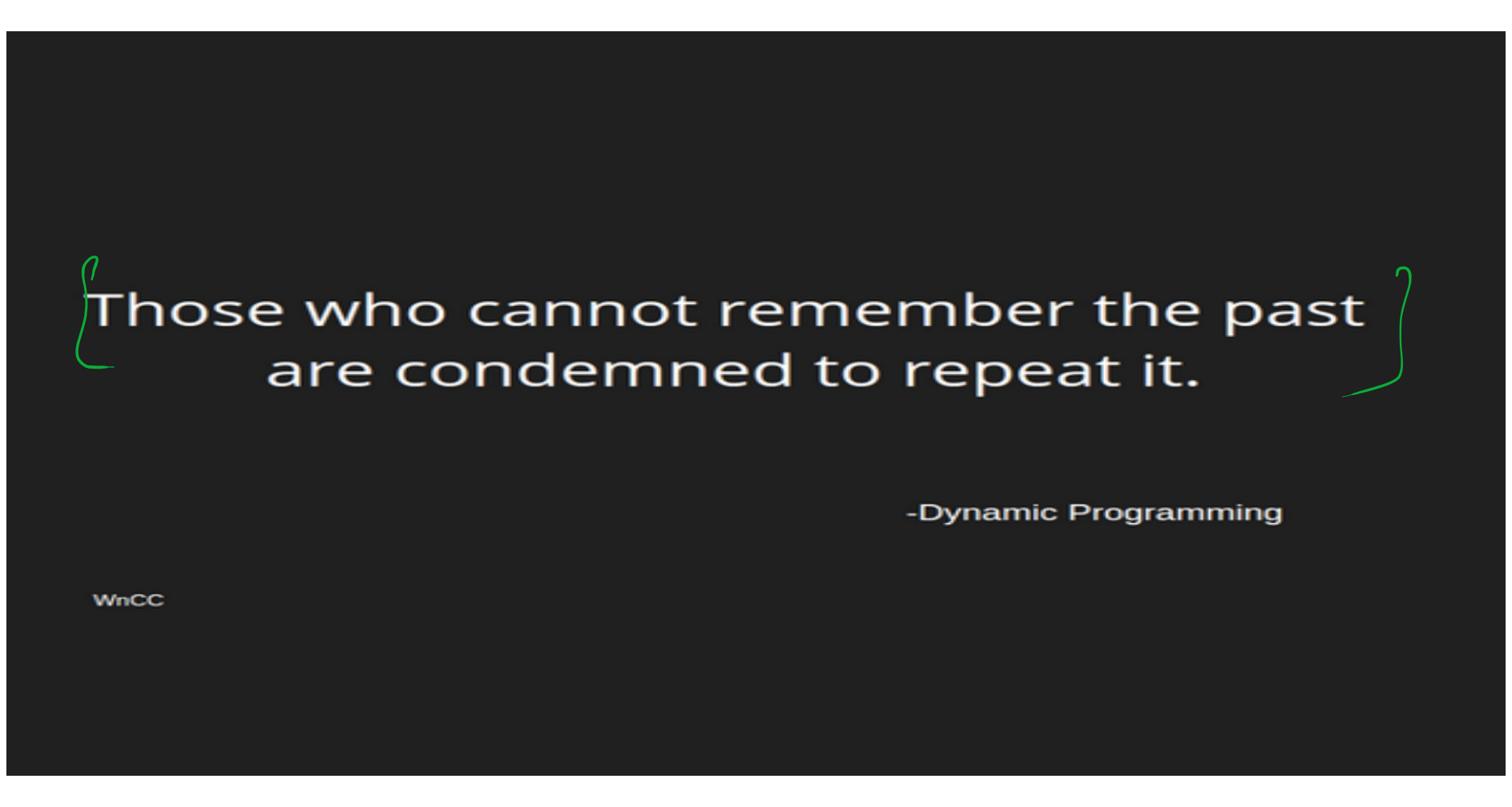
Lec-36

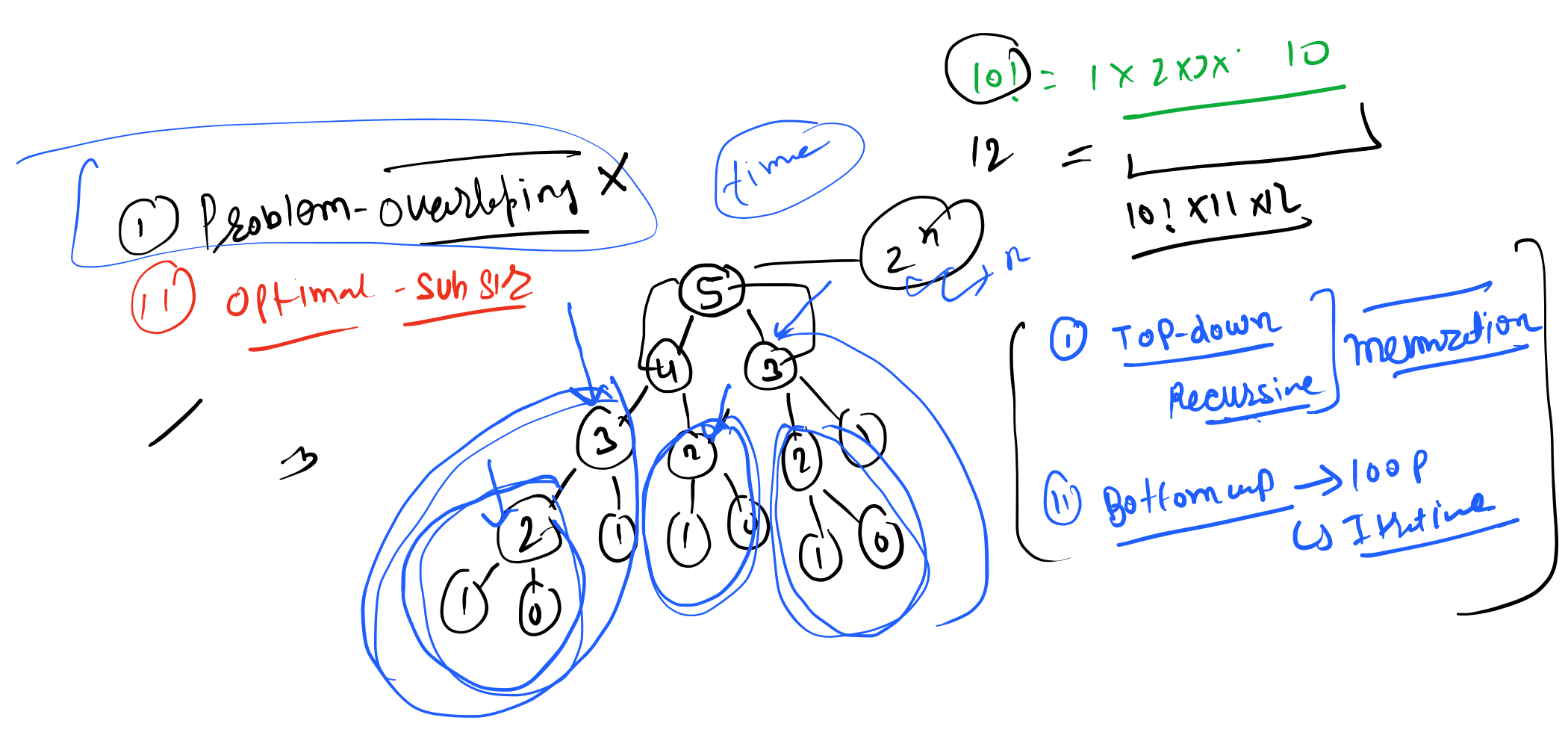
21/09/25 9:18 AM



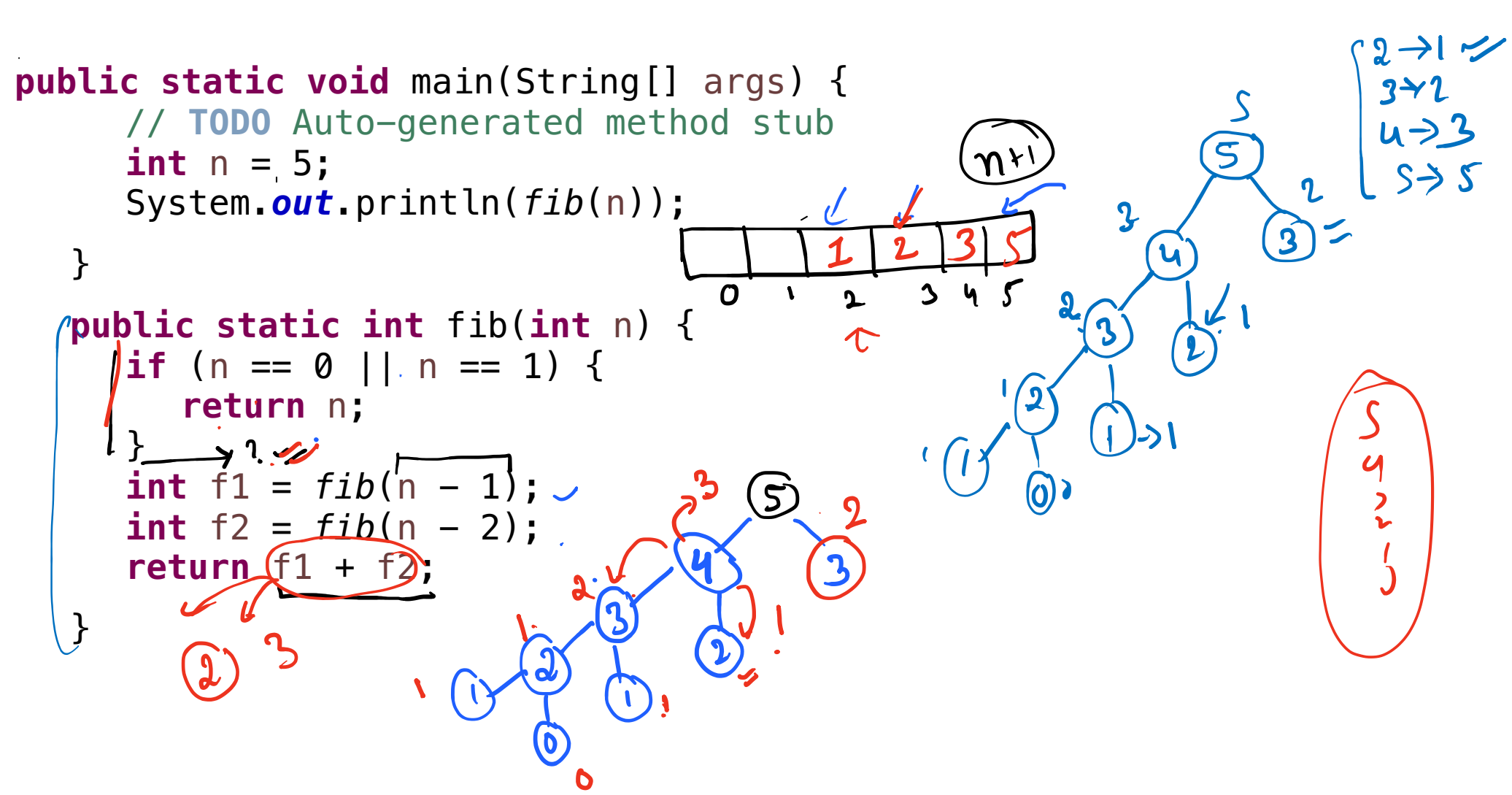
Those who cannot remember the past are condemned to repeat it.

-Dynamic Programming

WnCC

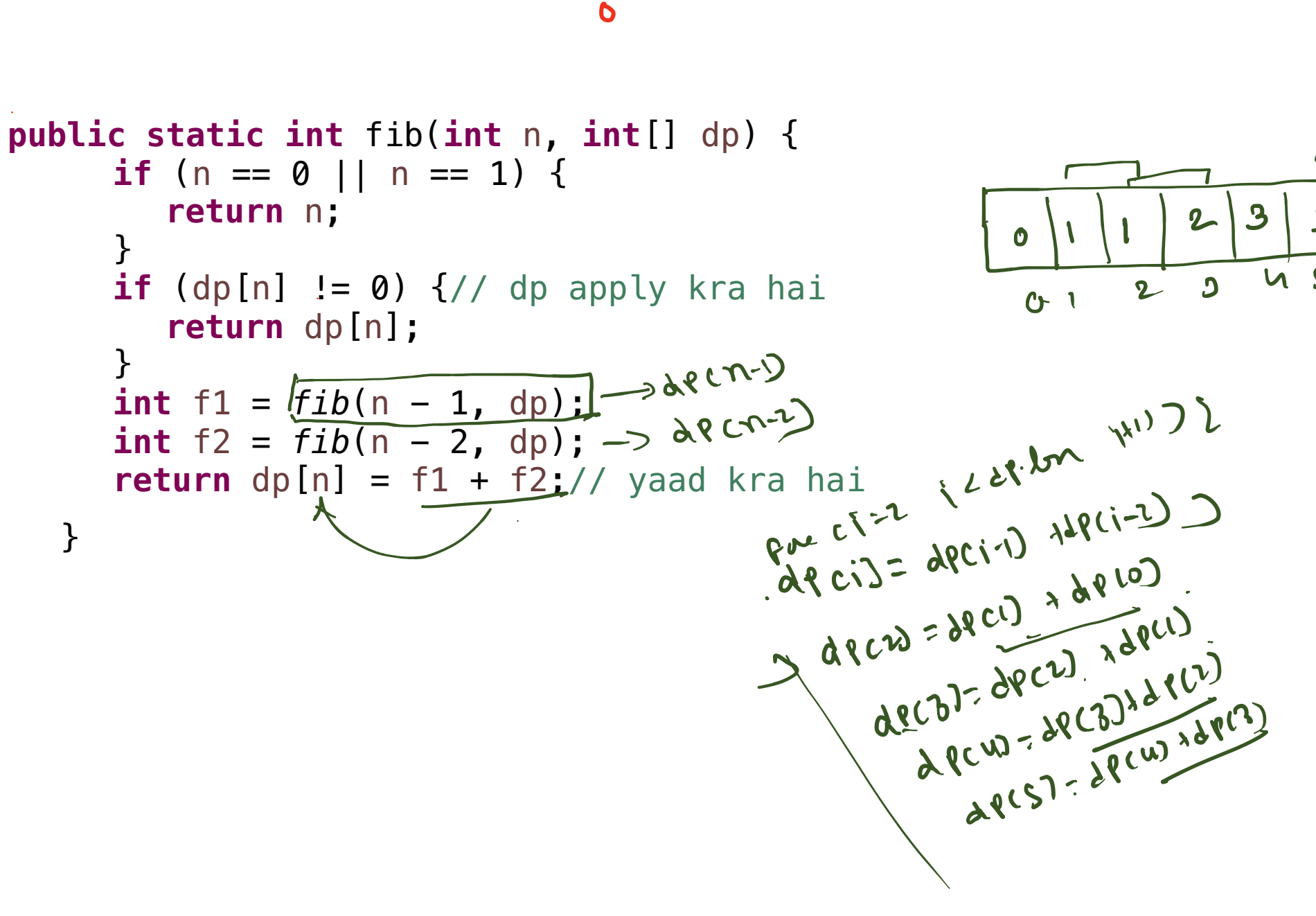


10! = 1 x 2 x 3 x ... 10
12 = 10! x 11 x 12
(i) Problem-overlapping
(ii) Optimal - Sub str
time
2^n
(i) Top-down Recursive
(ii) Bottom up → loop Iterative
memoization



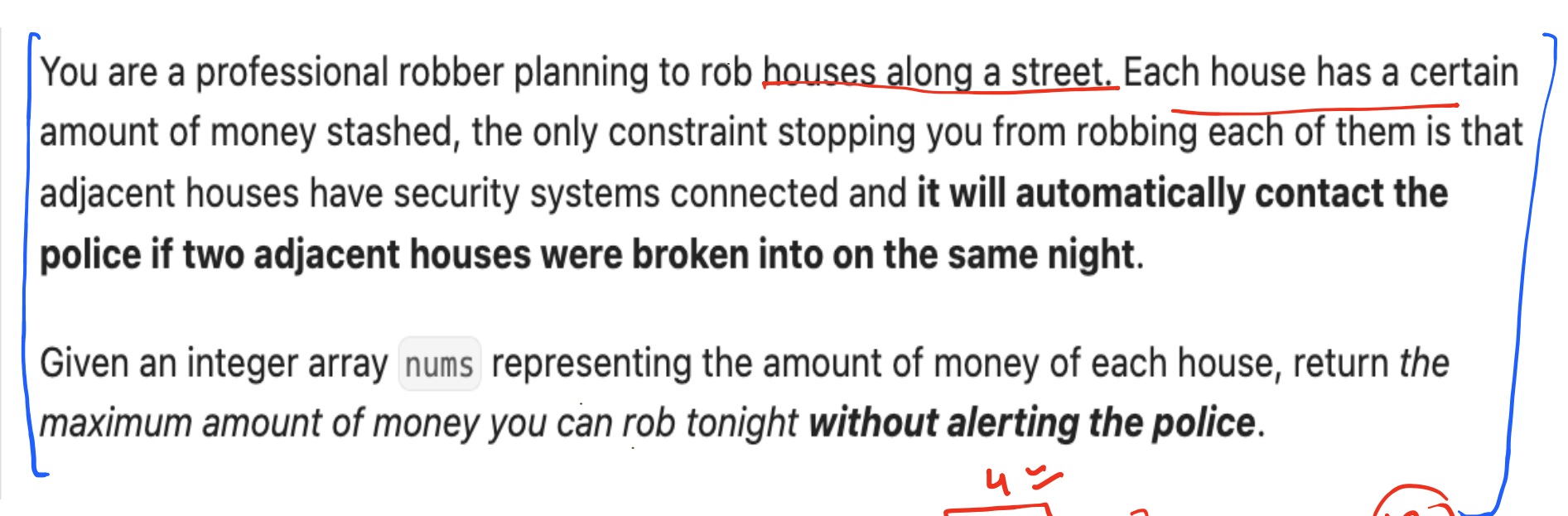
```
public static void main(String[] args) {
    // TODO Auto-generated method stub
    int n = 5;
    System.out.println(fib(n));
}
public static int fib(int n) {
    if (n == 0 || n == 1) {
        return n;
    }
    int f1 = fib(n - 1);
    int f2 = fib(n - 2);
    return f1 + f2;
}
```

n+1
0 1 2 3 4 5
2→1
3→2
4→3
5→5



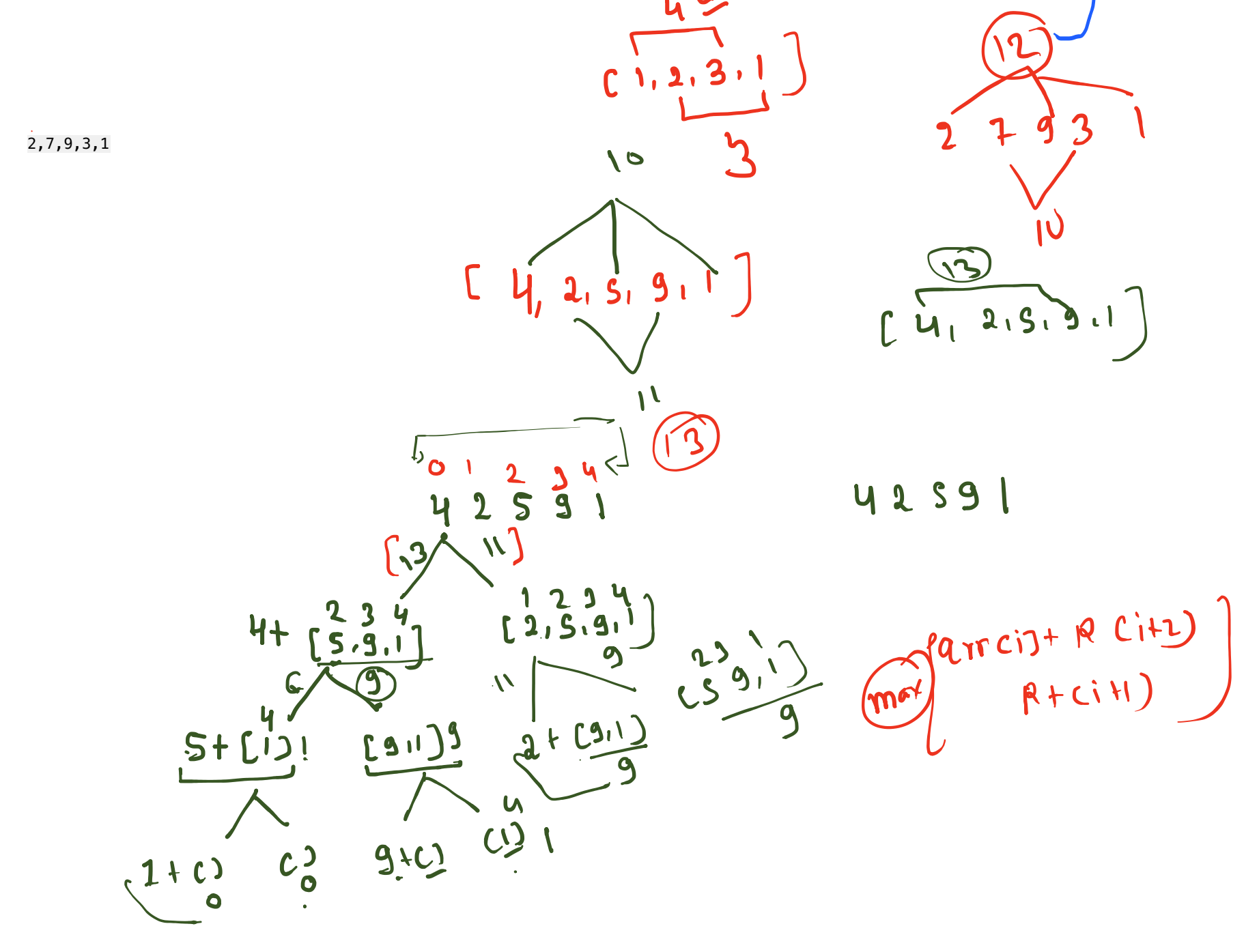
```
public static int fib(int n, int[] dp) {
    if (n == 0 || n == 1) {
        return n;
    }
    if (dp[n] != 0) {// dp apply kra hai
        return dp[n];
    }
    int f1 = fib(n - 1, dp);
    int f2 = fib(n - 2, dp);
    return dp[n] = f1 + f2;// yaad kra hai
}
```

→ dp(n-1)
→ dp(n-2)
0 1 1 2 3 5
0 1 2 3 4 5
dp[i] = dp[i-1] + dp[i-2]
dp(2) = dp(1) + dp(0)
dp(3) = dp(2) + dp(1)
dp(4) = dp(3) + dp(2)
dp(5) = dp(4) + dp(3)



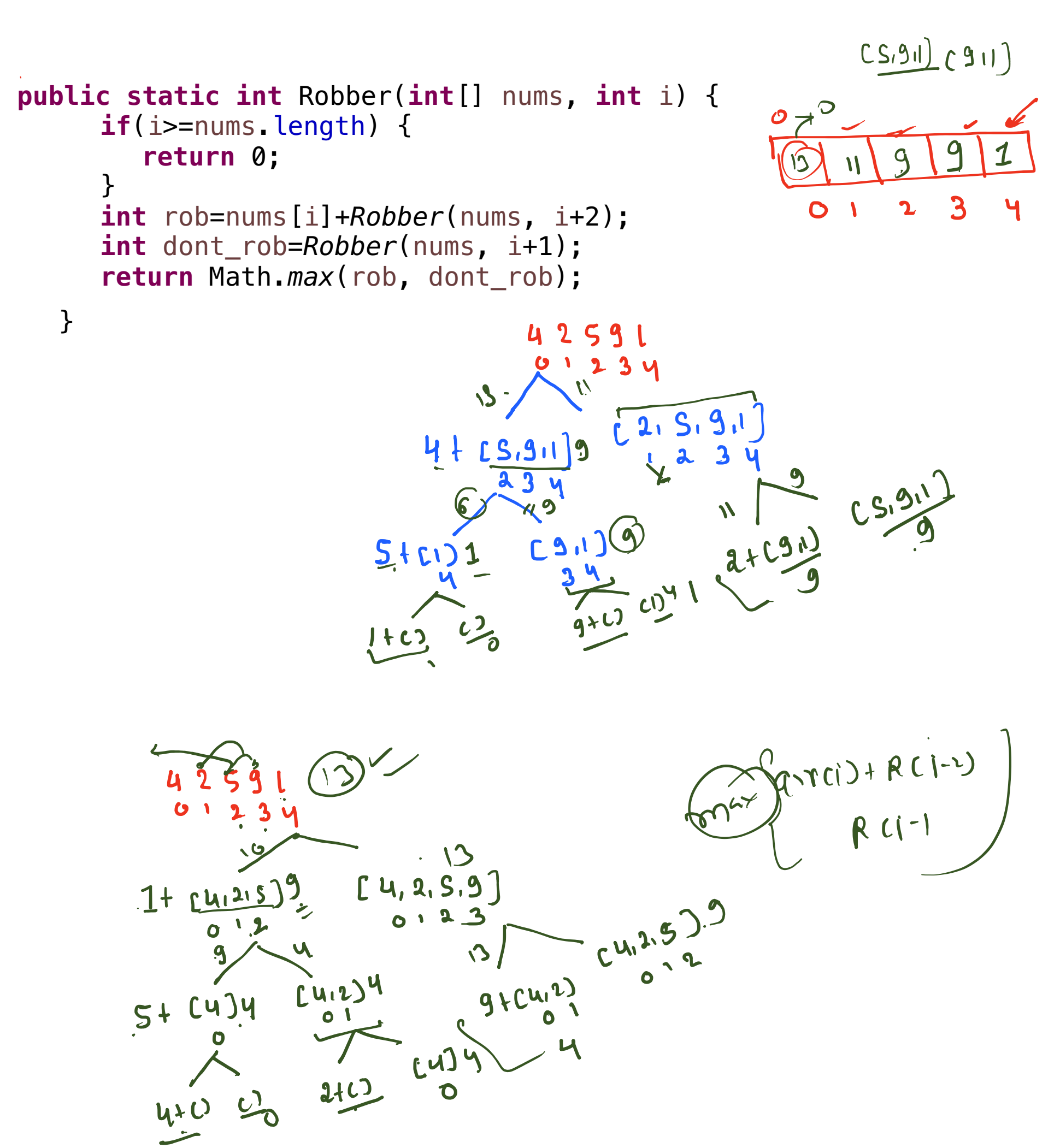
You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight* ***without alerting the police***.



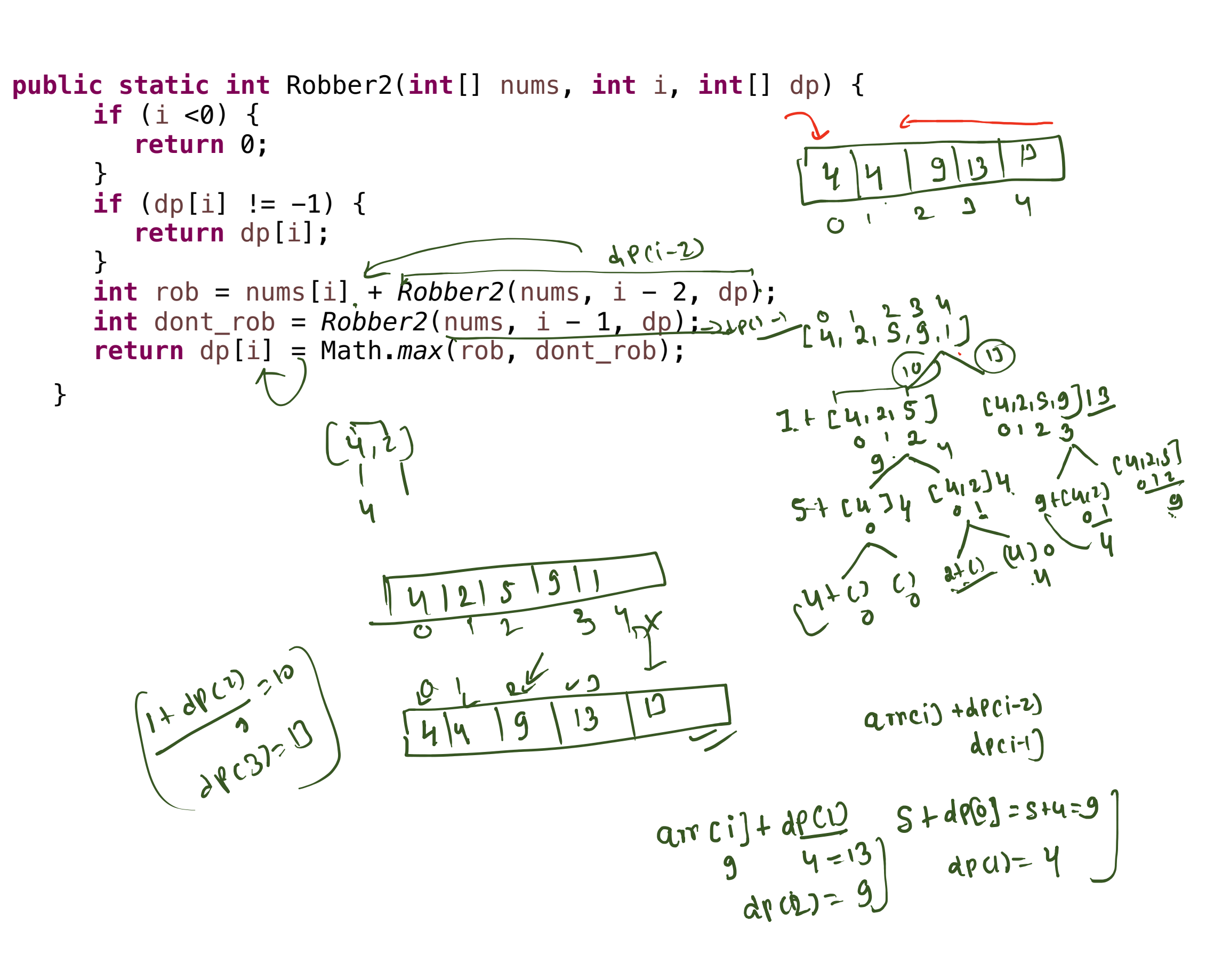
2,7,9,3,1

[1, 2, 3, 1]
[4, 2, 5, 9, 1]
4 2 5 9 1
max (arr[i] + R(i+2), R(i+1))



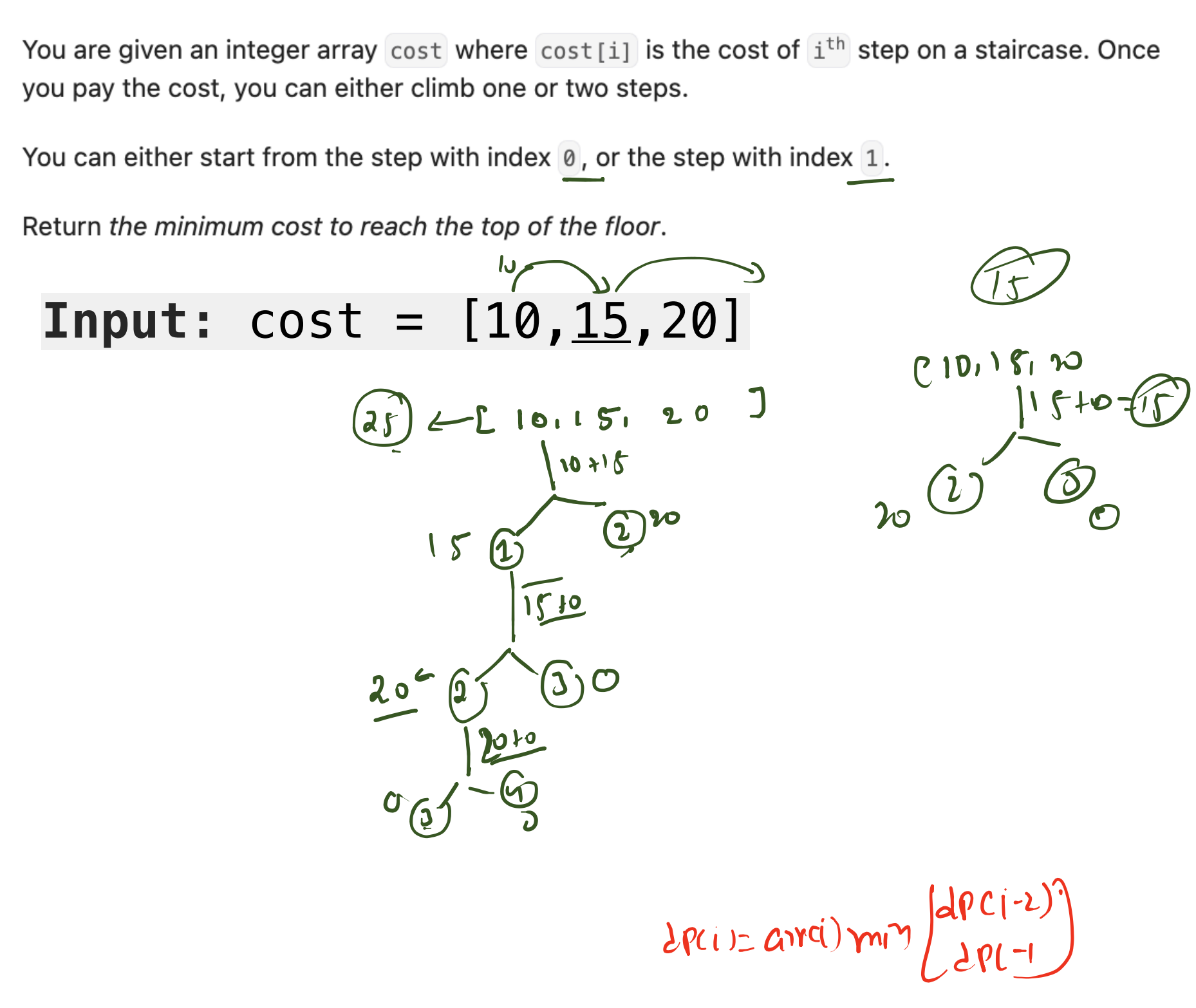
```
public static int Robber(int[] nums, int i) {
    if(i>=nums.length) {
        return 0;
    }
    int rob=nums[i]+Robber(nums, i+2);
    int dont_rob=Robber(nums, i+1);
    return Math.max(rob, dont_rob);
}
```

15 11 9 9 1
0 1 2 3 4
4 2 5 9 1
max (arr[i] + R(i-2), R(i-1))



```
public static int Robber2(int[] nums, int i, int[] dp) {
    if (i <0) {
        return 0;
    }
    if (dp[i] != -1) {
        return dp[i];
    }
    int rob = nums[i] + Robber2(nums, i - 2, dp);
    int dont_rob = Robber2(nums, i - 1, dp);
    return dp[i] = Math.max(rob, dont_rob);
}
```

4 4 9 13 13
0 1 2 3 4
4 1 2 5 9 1
arr[i] + dp[i-2]
dp[i-1]



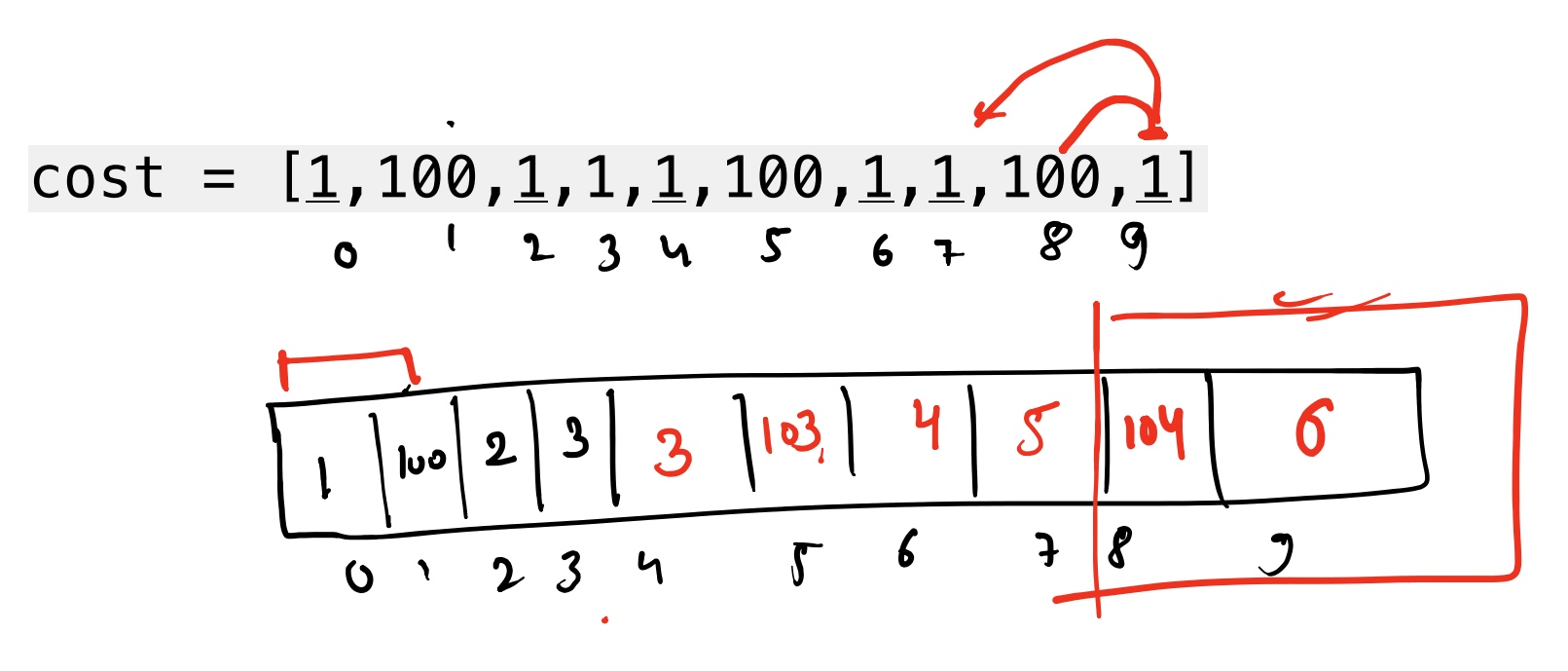
You are given an integer array `cost` where `cost[i]` is the cost of i^{th} step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index `0`, or the step with index `1`.

Return *the minimum cost to reach the top of the floor.*

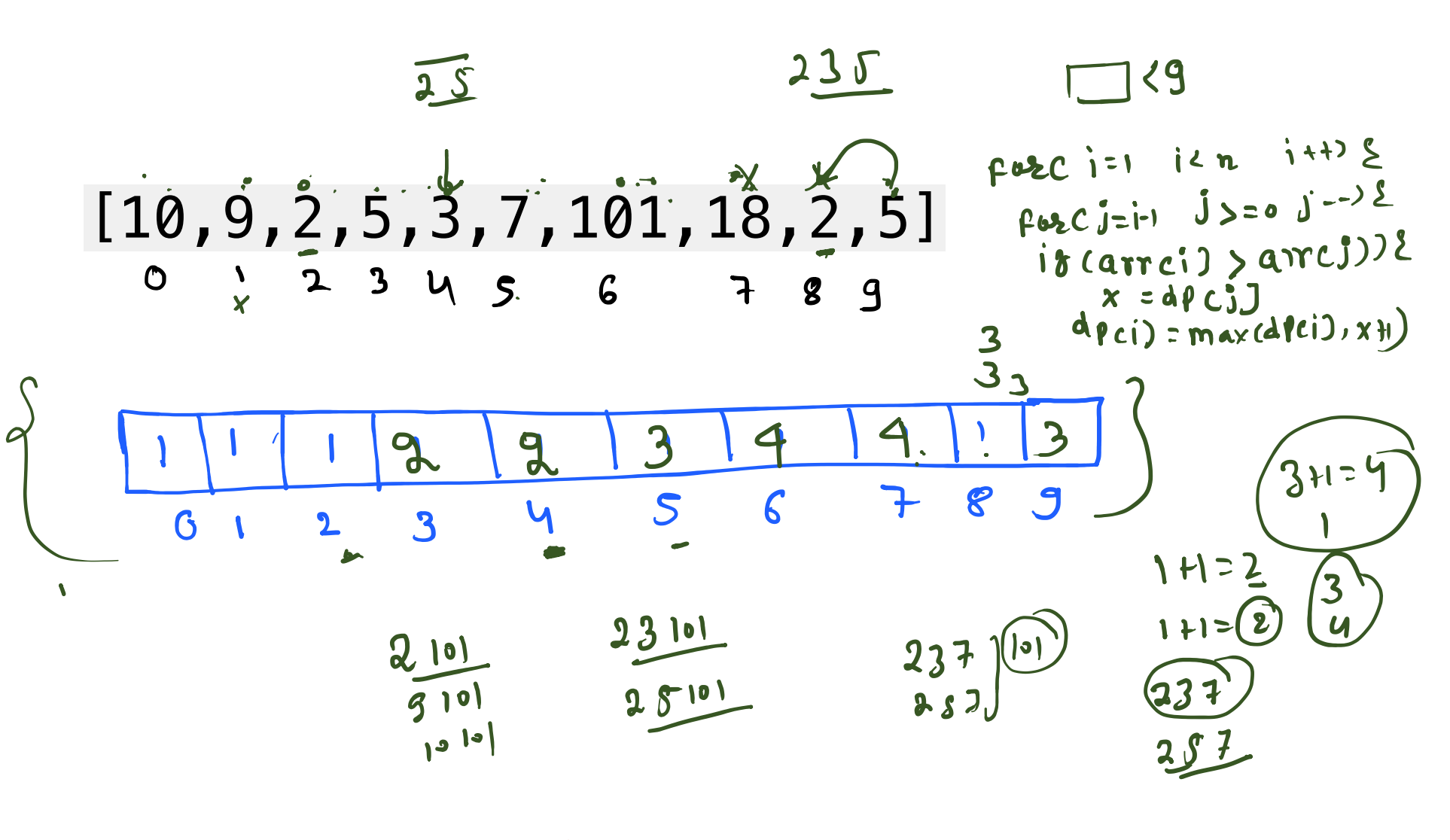
Input: cost = [10,15,20]

dp[i] = arr[i] + min (dp[i-2], dp[i-1])



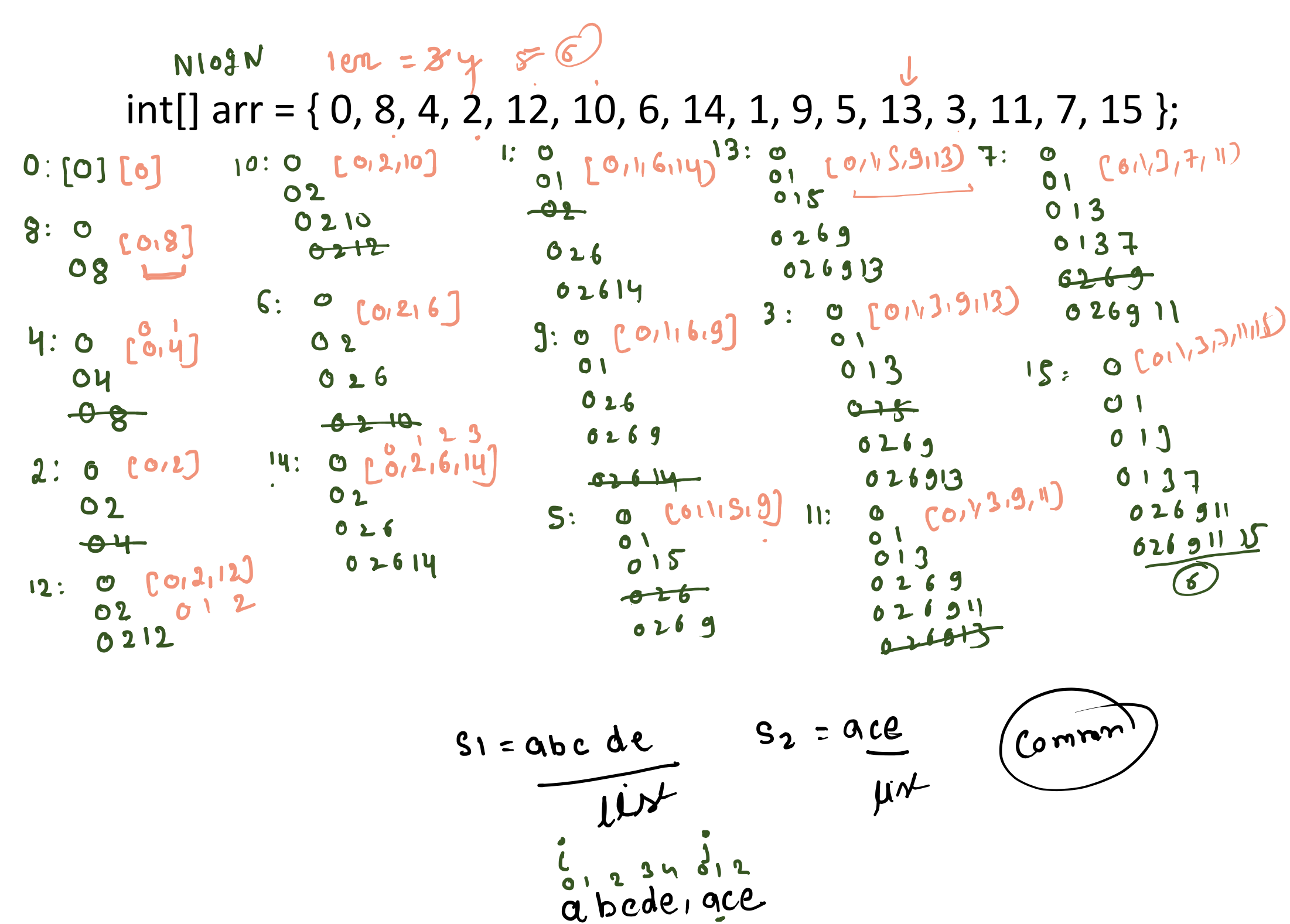
cost = [1,100,1,1,1,100,1,1,100,1]

1 100 2 3 3 103 4 5 104 6
0 1 2 3 4 5 6 7 8 9



[10,9,2,5,3,7,101,18,2,5]

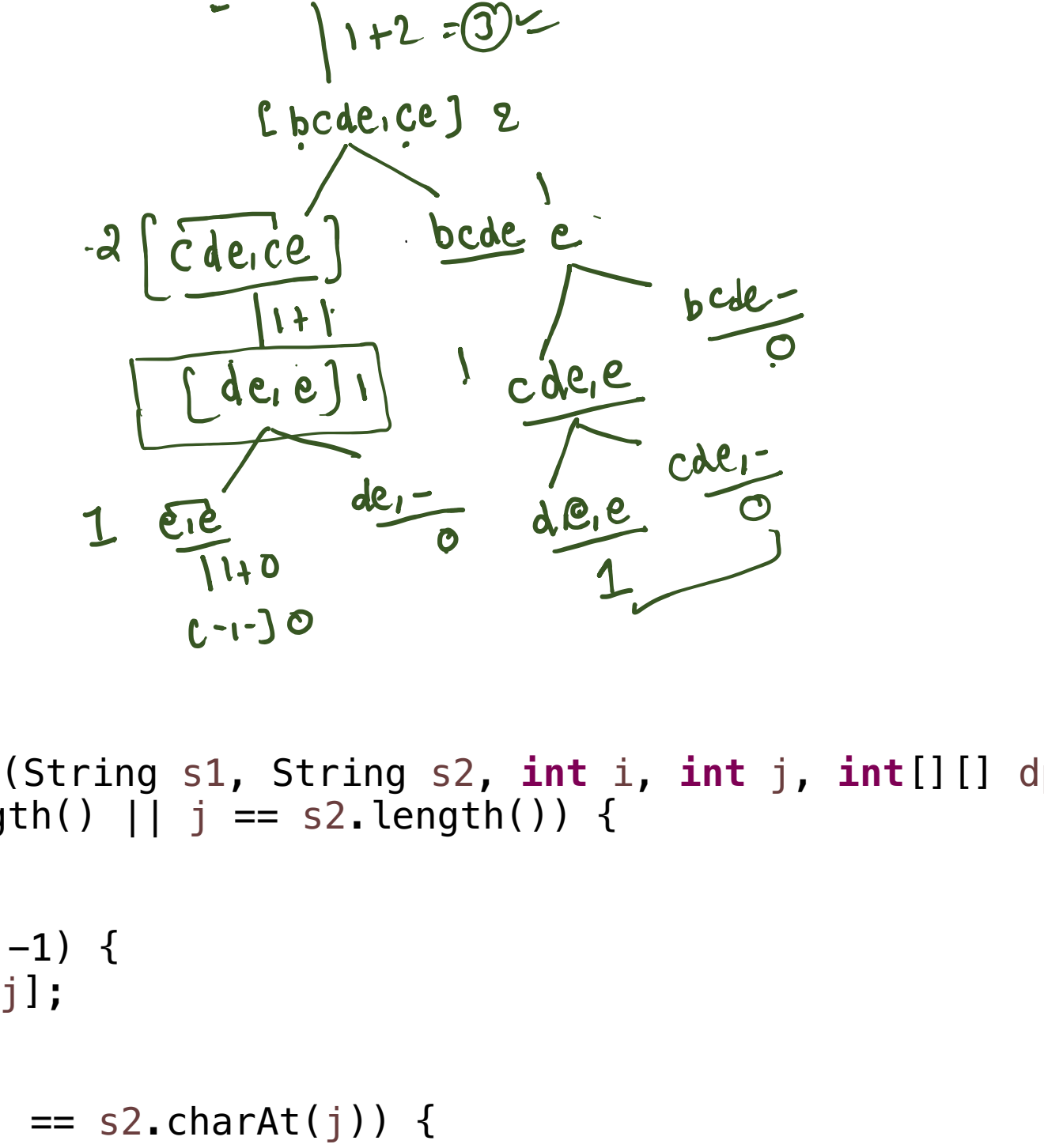
1 1 1 2 2 3 4 4 1 3
0 1 2 3 4 5 6 7 8 9
for(i=1 i<n i++){
for(j=i-1 j>=0 j--){
if(arr[i] > arr[j]){
x = dp[j]
dp[i] = max(dp[i], x+1)



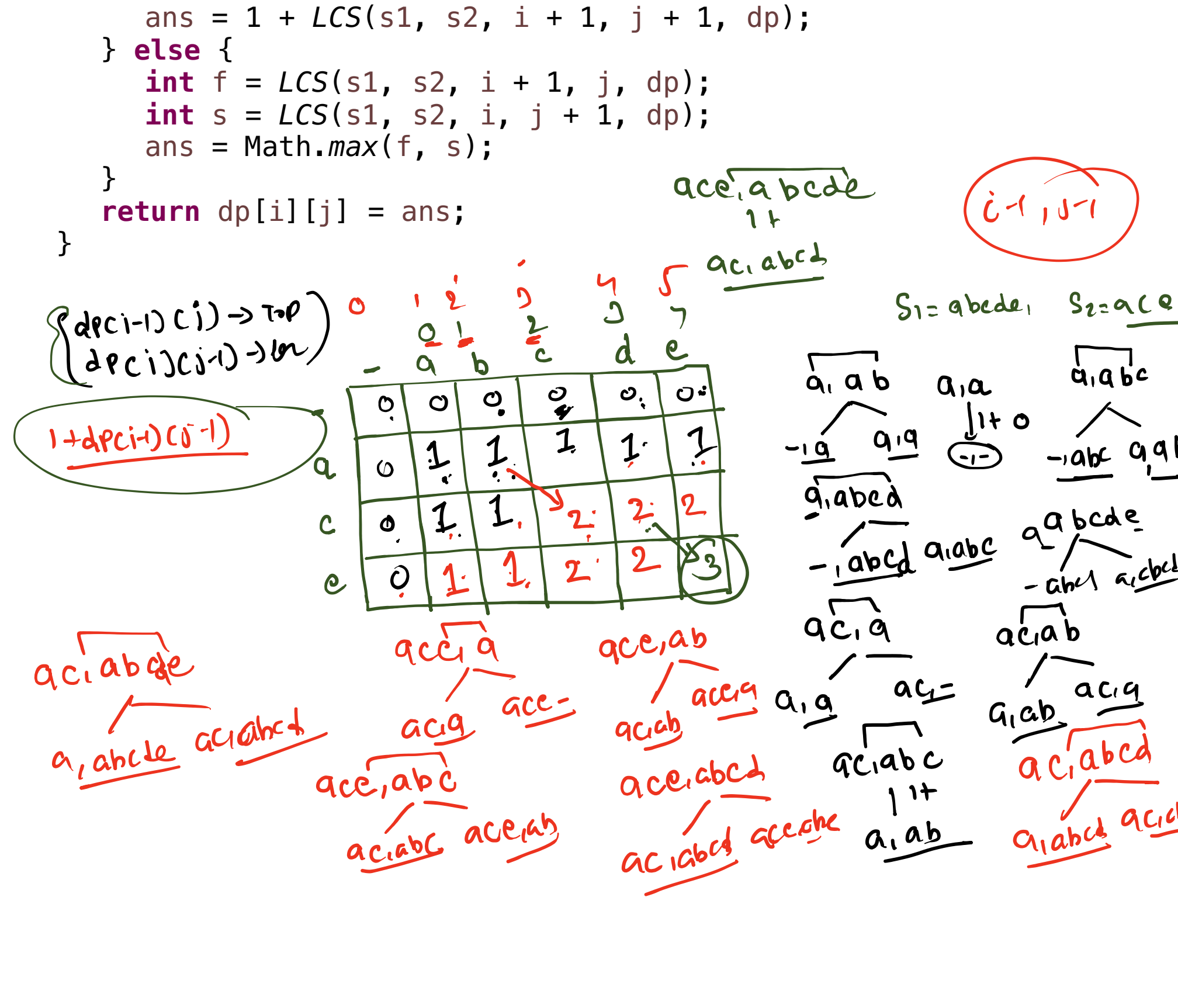
N log N

int[] arr = { 0, 8, 4, 2, 12, 10, 6, 14, 1, 9, 5, 13, 3, 11, 7, 15 };

len = 3
[0,1,3,7,11,15]
0 2 6 9 11 15
6

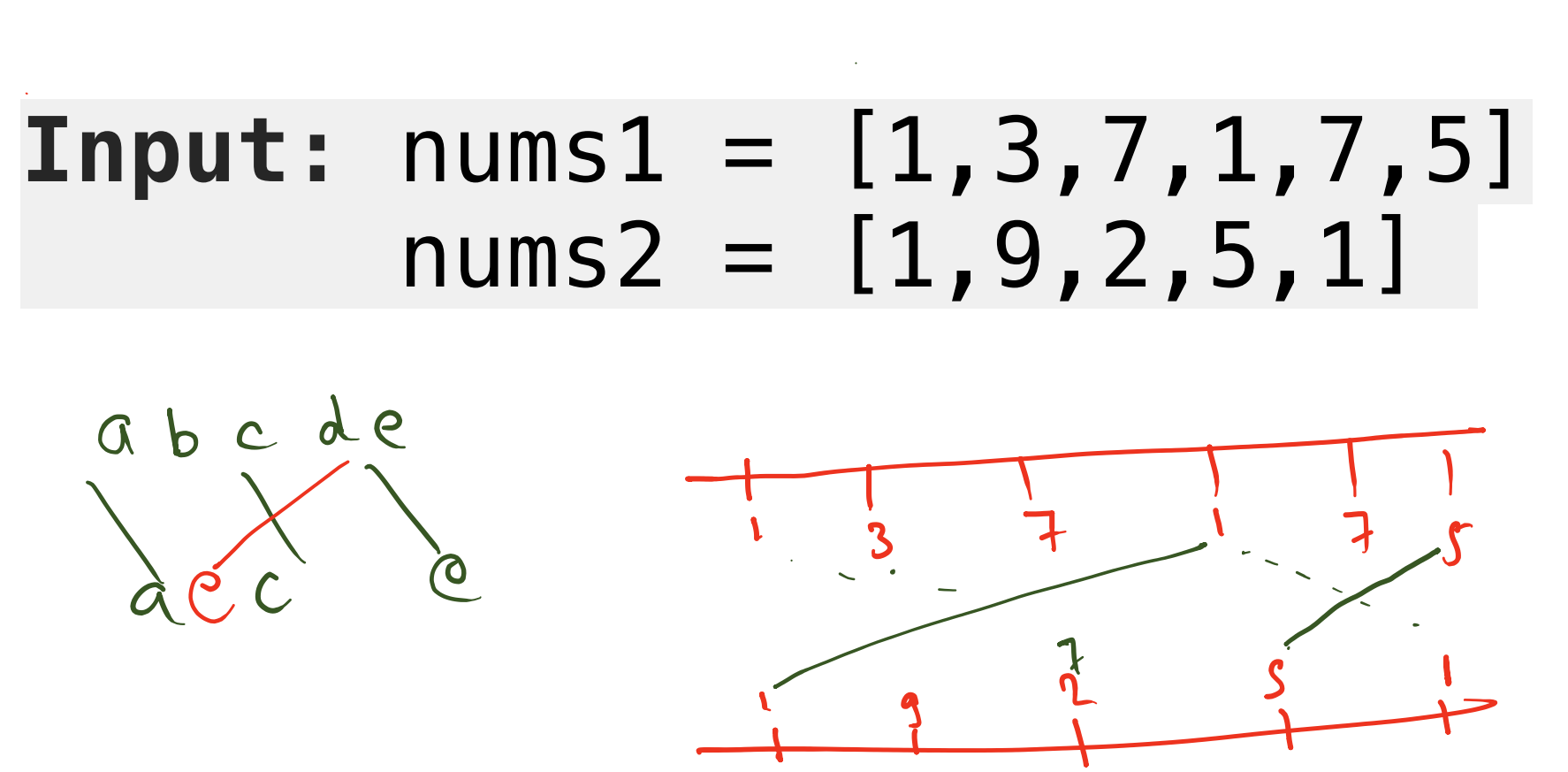


S1 = abcde
S2 = ace
Common
a bcde, ace
1+2 = 3
[bcde, ce] 2
[cde, ce]
[de, e]



```
public static int LCS(String s1, String s2, int i, int j, int[][] dp) {
    if (i == s1.length() || j == s2.length()) {
        return 0;
    }
    if (dp[i][j] != -1) {
        return dp[i][j];
    }
    int ans = 0;
    if (s1.charAt(i) == s2.charAt(j)) {
        ans = 1 + LCS(s1, s2, i + 1, j + 1, dp);
    } else {
        int f = LCS(s1, s2, i + 1, j, dp);
        int s = LCS(s1, s2, i, j + 1, dp);
        ans = Math.max(f, s);
    }
    return dp[i][j] = ans;
}
```

dp[i-1][j] → Top
dp[i][j-1] → left
1 + dp[i-1][j-1]
i-1, j-1
S1 = abcde, S2 = ace



Input: nums1 = [1,3,7,1,7,5]
nums2 = [1,9,2,5,1]